

完整版设计报告

<谢林珂> <计算机科学与技术> <3230105747>

1.项目背景

本项目是2025-2026学年《B/S体系软件设计》课程的综合性实践项目，旨在设计并实现一个现代化的图片管理网站。用户在注册登录后可以上传、管理、编辑和分享个人图片资源，系统能够自动提取图片的EXIF信息生成分类标签，并提供基于多种条件的智能检索功能，同时需要网站界面美观友好，样式适配移动端，能够在手机浏览器和微信等应用内置浏览器中良好显示。

本文档是项目的设计文档，包含了系统的需求分析，系统的总体架构设计，以及数据库设计、界面原型的设计等内容。本项目要求掌握完整的Web应用开发流程，从前端界面设计到后端业务逻辑实现，以及数据库设计与系统集成，通过实际项目开发加深对现代Web开发技术的理解与应用能力，并由一人独立完成全部开发工作。

2.系统需求设计

2.1 功能性需求分析

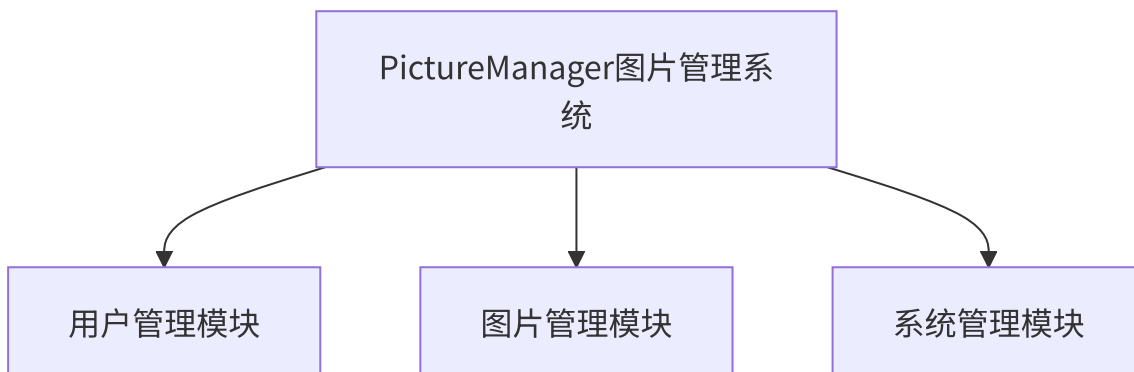
PictureManager图片管理网站是一个基于B/S架构的Web应用，主要面向需要管理个人或组织图片资源的用户。系统功能需求如下：

2.1.1用户角色定义

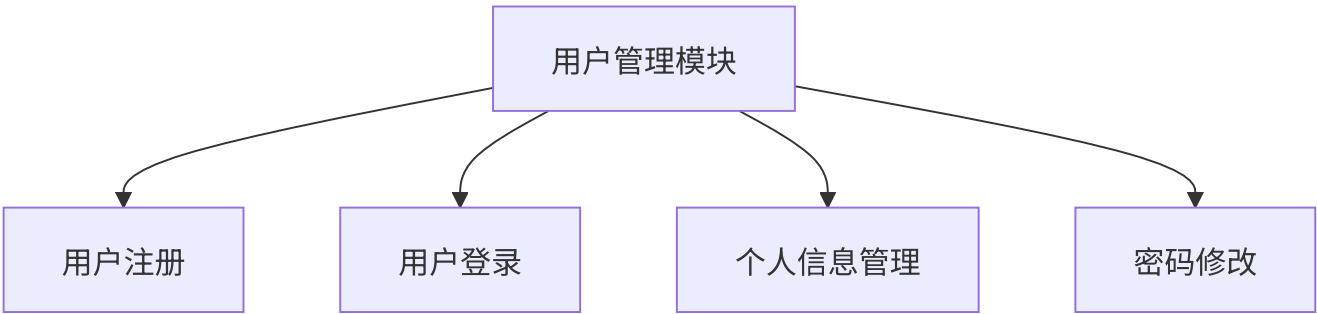
- 普通用户**：可以管理自己的图片资源，包括上传、编辑、分类、检索等操作
- 管理员用户**：除了普通用户的所有权限外，还可以管理用户账户、审核内容、查看系统统计等

2.1.2功能模块划分

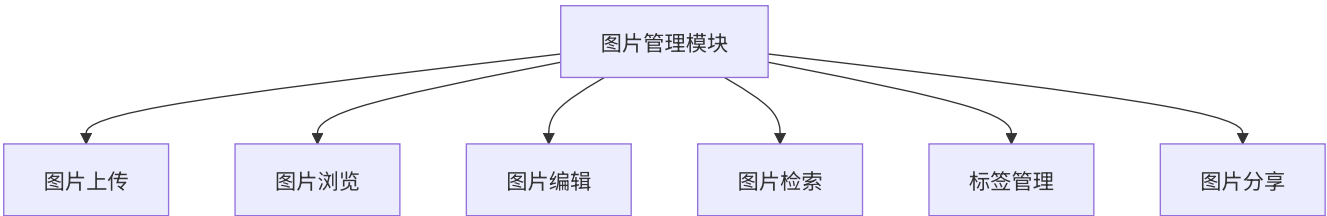
功能划分：



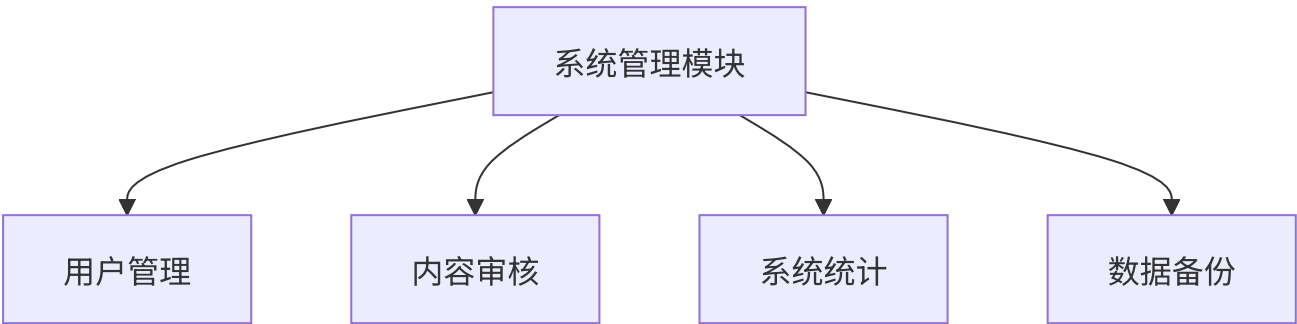
用户管理模块：



图片管理模块：



系统管理模块：



用户管理模块

- **用户注册**：需要用户名、密码、邮箱等关键信息，并进行有效性验证
- **用户登录**：支持多种登录方式（账号密码、第三方登录）
- **个人信息管理**：修改个人资料、头像、联系方式等
- **密码管理**：密码修改、重置密码功能

图片管理模块

- **图片上传**：支持单文件/批量上传，拖拽上传，自动生成缩略图
- **图片浏览**：多种视图模式（网格、列表），支持幻灯片播放
- **图片编辑**：在线裁剪、滤镜应用、色调调整等基础编辑功能
- **图片检索**：基于标签、分类、时间、EXIF信息等多条件智能检索
- **标签管理**：自动标签生成（基于EXIF和AI分析），自定义标签添加
- **图片分享**：生成分享链接，设置访问权限

系统管理模块（管理员）

- **用户管理**：用户账户审核、权限管理、账户封禁

- **内容审核：**图片内容审查，违规内容处理
- **系统统计：**用户活跃度、存储使用情况、热门标签等数据分析
- **数据备份：**定期数据备份与恢复机制

2.2 非功能性需求分析

2.2.1 性能需求

- **系统稳定性：**保证系统7×24小时稳定运行，月故障时间不超过1小时
- **浏览器兼容性：**支持Chrome、Firefox、Safari、Edge等主流浏览器
- **并发处理能力：**系统应能支持至少200人同时在线访问
- **响应时间：**
 - 页面加载时间：普通页面1秒内，图片列表页面3秒内
 - 用户操作响应时间：大部分操作应在1秒内完成反馈
 - 图片上传：支持大文件分块上传，提供进度显示
- **异常处理：**系统能及时检测网络异常、服务器错误等情况，提供友好错误提示
- **系统维护：**系统保证每月维护时间不超过4小时

2.2.2 输入输出需求

- **数据有效性检查：**
 - 用户注册时验证用户名唯一性、邮箱格式、密码强度
 - 图片上传时验证文件格式、大小限制
 - 所有表单输入进行前端和后端双重验证
- **数据展示要求：**
 - 图片以缩略图形式在列表中展示，支持懒加载
 - 统计信息以图表形式可视化呈现（如存储使用情况、上传趋势等）
 - 提供多种排序和筛选方式，界面直观易用
- **错误预防与处理：**
 - 通过输入验证、确认对话框等方式减少用户操作错误
 - 提供明确的操作反馈和错误提示信息
 - 实现操作撤销功能，减少误操作影响

2.2.3 数据管理需求

- **系统安全性与独立性：**
 - 实现基于角色的访问控制，防止未授权访问
 - 用户数据隔离，普通用户只能访问自己的图片资源

- 系统接口设计遵循最小权限原则
- 数据备份与恢复：
 - 每日自动备份用户数据和图片元信息
 - 提供数据导出功能，支持用户备份个人数据
 - 系统故障时能在2小时内恢复服务
- 数据安全：
 - 用户密码加密存储，使用加盐哈希算法
 - 敏感数据传输使用HTTPS加密
 - 实现会话管理，支持安全退出
 - 定期进行安全漏洞扫描和修复

2.2.4 移动端适配需求

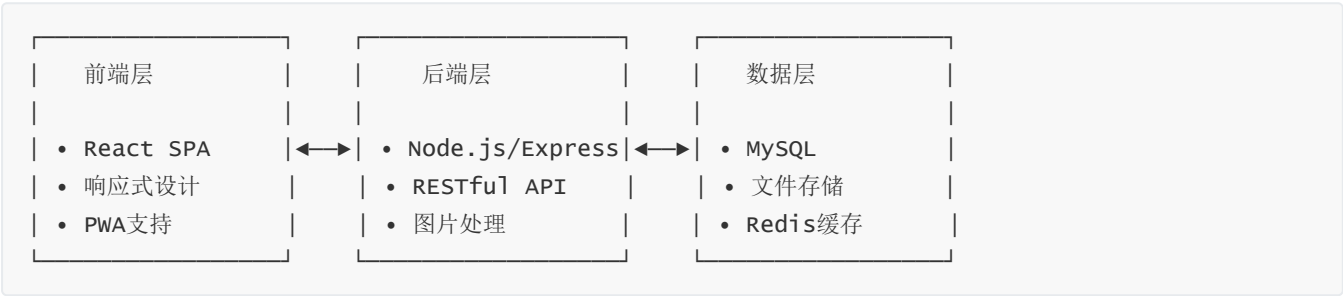
- 响应式设计：界面自动适配不同屏幕尺寸，从手机到桌面设备
- 移动端优化：
 - 触摸友好的交互设计，支持手势操作
 - 移动端上传优化，支持相机直接拍摄上传
 - 移动端数据流量优化，图片按需加载
- 跨平台兼容：在微信内置浏览器、手机自带浏览器等环境中正常使用

2.2.5 扩展性需求

- 模块化设计：各功能模块松耦合，便于功能扩展和维护
- API接口：提供RESTful API，支持第三方应用集成
- AI功能扩展：预留AI图像分析接口，便于后续功能增强
- 存储扩展：支持多种存储后端（本地存储、云存储）

3.系统技术架构

3.1 架构概述



这是一个基于React+Node.js的现代化全栈图片管理网站架构。前端使用React构建响应式单页面应用，提供流畅的用户交互体验和PWA支持，确保在手机和桌面设备上都能完美显示。后端采用Node.js和Express框架搭建RESTful API服务，处理用户认证、图片上传、EXIF信息提取和业务逻辑。数据层使用MySQL存储结构化数据（用户信息、图片元数据），Redis缓存热点数据提升性能，文件系统存储原始图片和缩略图。整个架构实现了前后端分离，通过API进行数据交互，具备良好的可扩展性和维护性。

3.2 技术栈选择

- 前端：React + Vite + React Router + 自定义CSS组件 + Font Awesome图标库 + 原生JavaScript交互
- 后端：Node.js + Express框架 + Multer文件上传 + Sharp图片处理 + JWT认证 + bcrypt密码加密
- 数据库：MySQL + Redis缓存
- 图片处理：Sharp缩略图生成 + exifr EXIF信息提取
- AI大模型功能：阿里云百炼APIKey
- 构建工具：Vite快速构建 + 原生JavaScript模块化

4.数据库设计

4.1 MySQL表结构

users表：

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  avatar_url VARCHAR(255),  
  storage_used BIGINT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

images表

相比中期修改了，增加了独立的EXIF查询列。

```
CREATE TABLE images (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  title VARCHAR(255),  
  description TEXT,  
  original_filename VARCHAR(255),  
  file_path VARCHAR(500) NOT NULL,  
  thumbnail_path VARCHAR(500),  
  file_size INT NOT NULL,
```

```

mime_type VARCHAR(50),
width INT,
height INT,
-- 核心: 提取EXIF关键字段方便查询
captured_at DATETIME,          -- 拍摄时间
location_address VARCHAR(255), -- 地理位置(解析后的字符串)
exif_data JSON,                -- 原始完整JSON
privacy ENUM('public', 'private') DEFAULT 'private',
upload_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

```

tags表

```

CREATE TABLE tags (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL,
  type ENUM('auto', 'manual') DEFAULT 'manual'
);

```

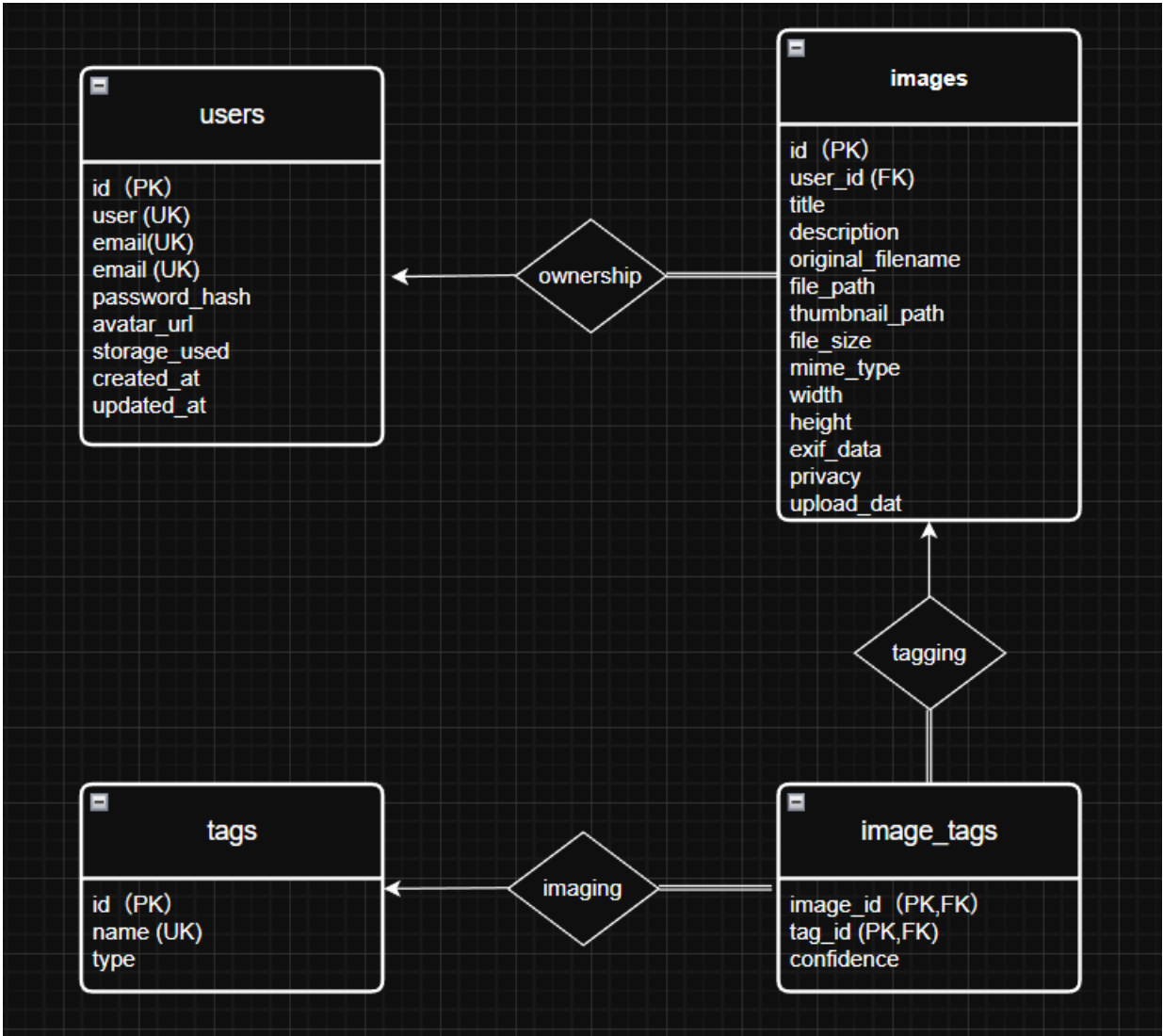
image_tags表

```

CREATE TABLE image_tags (
  image_id INT NOT NULL,
  tag_id INT NOT NULL,
  confidence FLOAT DEFAULT 1.0, -- AI标签的置信度
  PRIMARY KEY (image_id, tag_id),
  FOREIGN KEY (image_id) REFERENCES images(id) ON DELETE CASCADE,
  FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
);

```

4.2 ER图



5.系统接口设计

简单介绍所有核心的后端接口，具体见附件代码。

5.1 用户认证相关接口

用户认证类接口主要处理用户注册、登录和身份验证，提供的接口URL均以/api/auth开头。

5.1.1 用户注册接口

接口URL	POST /api/auth/register
主要参数列表	{ "username": "string, 最少6个字符", "email": "string, 有效邮箱格式", "password": "string, 最少6个字符" }
返回值类型	{ "message": "string, 注册结果提示", "code": 201 // 成功状态码 } 或 { "message": "string, 错误描述", "code": 400 // 客户端错误 }

接口简介	用于新用户注册，进行用户名和邮箱的唯一性校验，对密码进行bcrypt加密存储，返回注册结果。
------	--

接口核心逻辑流程：

- 接收客户端提交的用户名、邮箱、密码
- 进行非空校验和长度验证（≥6字符）
- 查询数据库检查用户名和邮箱唯一性
- 使用bcrypt对密码进行哈希加密（成本因子10）
- 将用户信息插入users表
- 返回201状态码和成功信息

5.1.2 用户登录接口

接口URL	POST /api/auth/login
主要参数列表	{ "email": "string, 注册时使用的邮箱", "password": "string, 用户密码" }
返回值类型	{ "message": "string, 登录结果提示", "token": "string, JWT令牌(24小时有效期)", "user": { "id": "number, 用户ID", "username": "string, 用户名", "email": "string, 用户邮箱" } } 或错误响应： { "message": "邮箱或密码错误", "code": 401 }
接口简介	用于用户登录认证，验证邮箱和密码匹配后生成JWT令牌返回给客户端，客户端需在后续请求的Authorization头中携带此令牌。

安全设计要点：

- 统一错误提示，"邮箱或密码错误"（避免泄露具体错误信息）
- JWT令牌包含userId和username，有效防止篡改
- 密码比较使用bcrypt.compare()，防止时序攻击

5.2 图片管理相关接口

图片管理类接口处理图片的上传、查询、编辑、删除和标签管理，提供的接口URL均以/api/images开头。所有接口都需要JWT身份验证。

5.2.1 全局认证中间件(middleware/auth.js)

中间件实现思路：

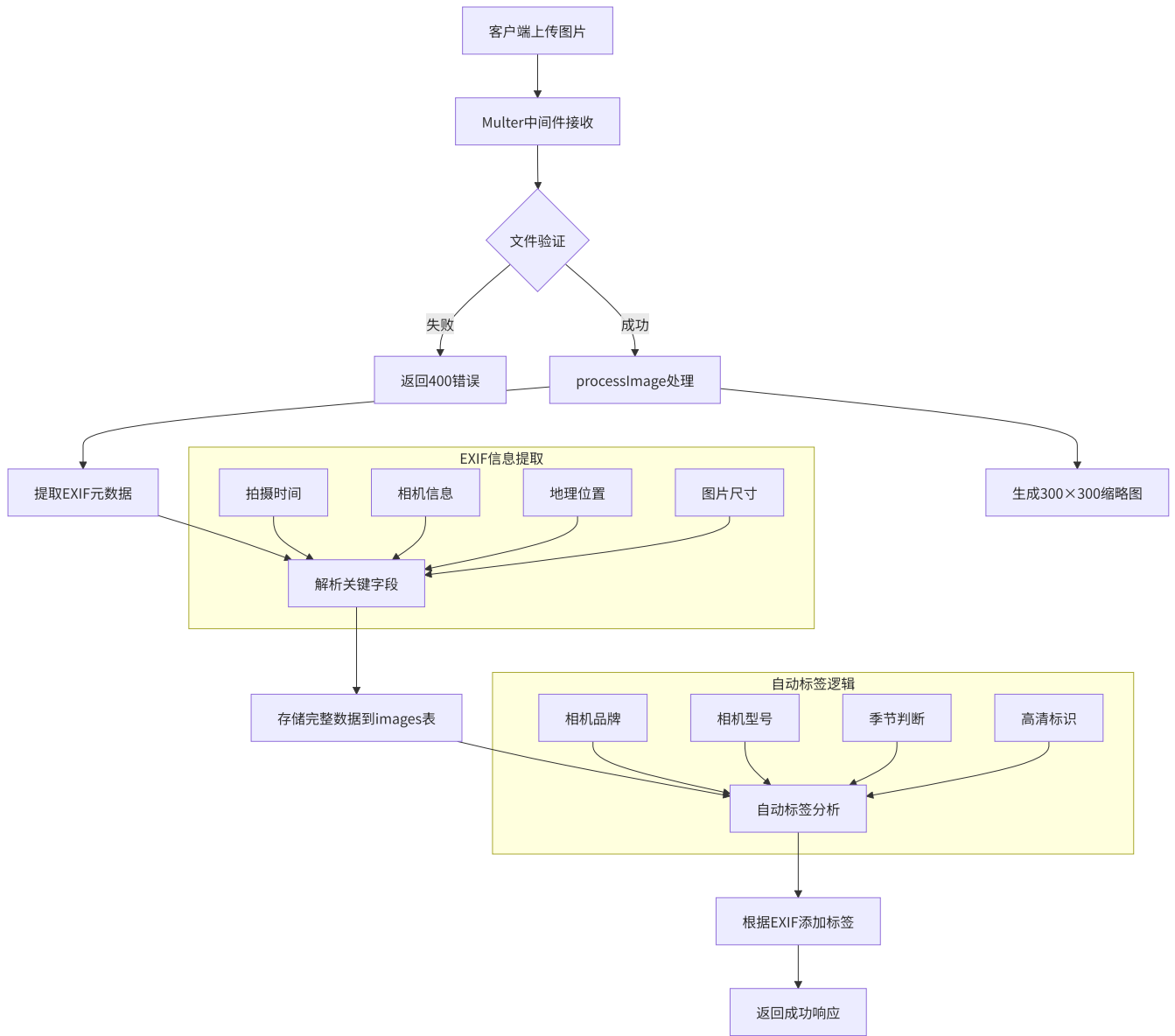
- 3. 从Authorization头提取Bearer令牌
- 4. 使用JWT_SECRET验证令牌有效性
- 5. 令牌解码后包含: { userId, username }
- 6. 将用户信息注入req.user供后续中间件使用
- 7. 无效/过期令牌返回403错误

具体见附件代码。

5.2.2 图片上传接口

接口URL	POST /api/images/upload			
请求格式	multipart/form-data			
主要参数列表	参数名	类型	必填	描述
	image	File	是	图片文件 (≤5MB, 支持image/*格式)
	title	String	否	图片标题, 默认使用文件名
	description	String	否	图片描述
	privacy	String	否	隐私设置: 'public'或'private', 默认'private'
返回值类型	{ "message": "图片上传并处理成功! ", "imageId": 1, "thumbnail": "uploads/thumbnails/img-1234567890.jpg", "exif": { "DateTimeOriginal": "2024:01:15 10:30:25", "Make": "Apple", "Model": "iPhone 13", "latitude": 30.2741, "longitude": 120.1551 } }			
接口简介	核心上传接口, 处理图片文件接收、EXIF信息提取、缩略图生成、自动标签分析和数据库存储。			

处理流程示意图:



数据库字段映射关系：

images表字段	数据来源	处理逻辑
user_id	req.user.userId	从JWT令牌解析
title	req.body.title 或 req.file.originalname	优先使用用户输入
file_path	req.file.path	Multer存储路径
thumbnail_path	processImage 返回	300×300缩略图路径
exif_data	EXIF提取结果	JSON格式完整存储
width/height	EXIF或Sharp分析	原始图片尺寸
captured_at	exif.DateTimeOriginal	拍摄时间
location_address	exif.latitude/longitude	GPS坐标格式化

5.2.3 图片查询接口

接口URL	GET /api/images		
查询参数	参数名	类型	描述
	title	String	按标题模糊搜索（LIKE %title%）
	startDate	Date	拍摄时间起始范围（与endDate同时使用）
	endDate	Date	拍摄时间结束范围
	privacy	String	隐私筛选：'public'或'private'
返回值类型	[{ "id": 1, "title": "西湖日落", "description": "美丽的西湖日落景色", "file_path": "uploads/original/img-1234567890.jpg", "thumbnail_path": "uploads/thumbnails/thumb-1234567890.jpg", "width": 4032, "height": 3024, "captured_at": "2024-01-15T10:30:25.000Z", "location_address": "30.274100, 120.155100", "privacy": "private", "upload_date": "2024-12-01T08:15:30.000Z" }]		
接口简介	查询当前用户的所有图片，支持多种筛选条件，按上传时间倒序排列。		

SQL查询构建逻辑：

```
-- 基础查询：用户隔离
SELECT * FROM images WHERE user_id = ?

-- 条件动态拼接
IF title: AND title LIKE ?
IF dateRange: AND captured_at BETWEEN ? AND ?
IF privacy: AND privacy = ?

-- 排序
ORDER BY upload_date DESC
```

5.2.4 图片删除接口

接口URL	DELETE /api/images/:id
路径参数	• :id- 图片ID（必填）
返回值类型	{ "message": "图片已成功删除" }

接口简介	删除指定图片，包含数据库记录删除和物理文件清理，确保数据一致性。
------	----------------------------------

删除流程安全设计：

- 权限验证：WHERE id = ? AND user_id = ?防止越权删除
- 物理文件清理：先查询文件路径，再删除原图和缩略图
- 异常处理：文件不存在时记录日志但不中断流程
- 事务性：确保数据库记录删除成功

5.2.5 标签管理接口

5.2.5.1 添加标签接口

接口URL	POST /api/images/:id/tags
请求体	{ "tagName": "string, 标签名称" }
返回值类型	{ "message": "标签添加成功", "tagName": "风景" }
接口简介	为指定图片添加人工标签，自动处理标签去重和关联关系建立。

标签处理逻辑：

```
// 1. 检查标签是否已存在
SELECT id FROM tags WHERE name = ?

// 2. 不存在则创建新标签
INSERT INTO tags (name) VALUES (?)

// 3. 检查图片-标签关联是否已存在
SELECT * FROM image_tags WHERE image_id = ? AND tag_id = ?

// 4. 不存在则建立关联
INSERT INTO image_tags (image_id, tag_id) VALUES (?, ?)
```

5.2.5.2 取图片标签接口

接口URL	GET /api/images/:id/tags
返回值类型	[{ "name": "风景" }, { "name": "iPhone拍摄" }, { "name": "夏天" }]
接口简介	获取指定图片的所有标签列表，用于前端展示和编辑。

5.2.6 图片编辑接口

接口URL	POST /api/images/:id/edit
请求格式	multipart/form-data
主要参数	image- 新的图片文件（替换原图）
返回值类型	{ "message": "编辑成功", "file_path": "uploads/original/img-9876543210.jpg" }
接口简介	替换图片文件，自动生成新缩略图，更新数据库中的文件信息和尺寸数据。

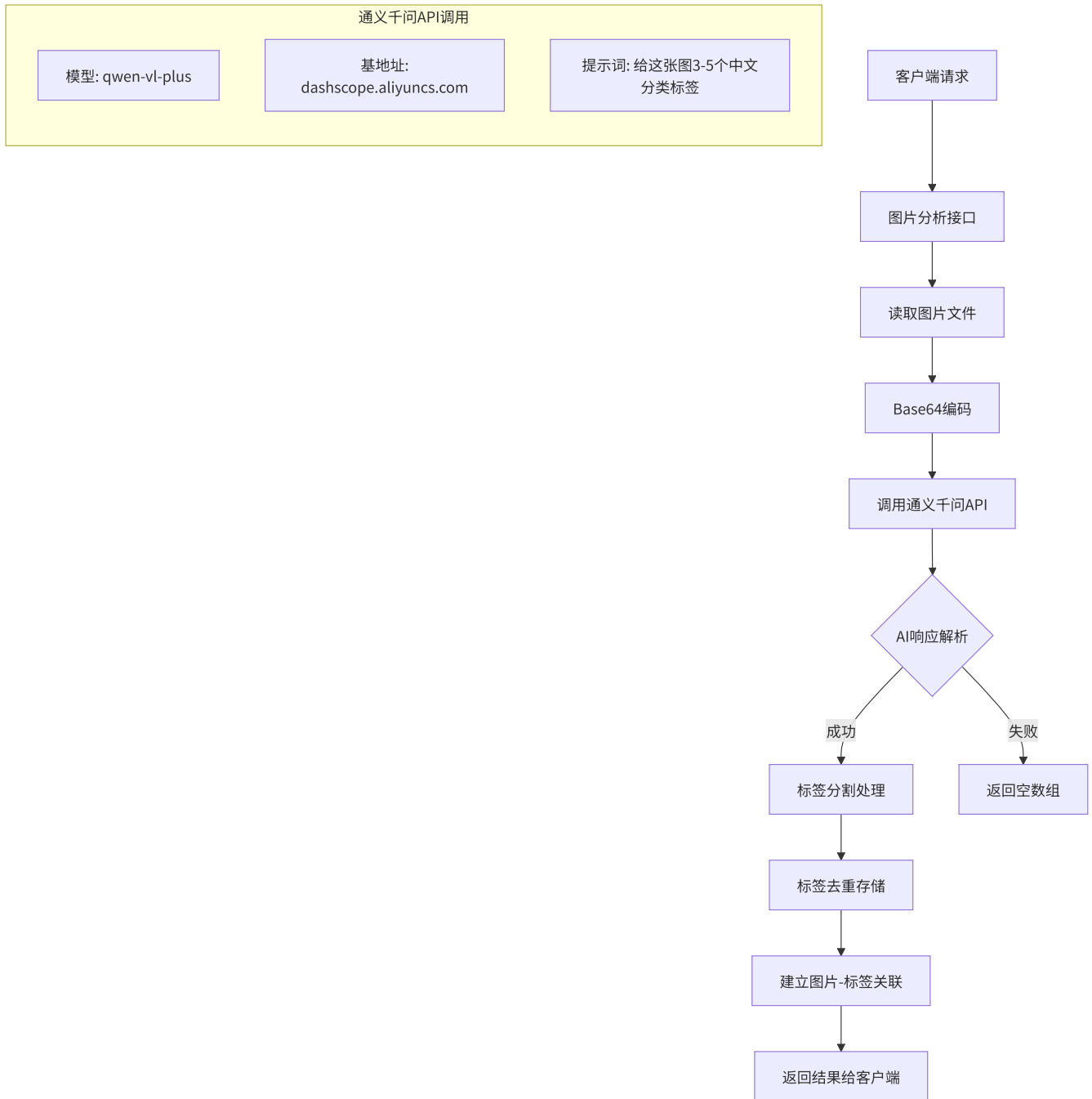
编辑处理流程：

- 权限验证：确保用户只能编辑自己的图片
- 新文件处理：使用Sharp生成300×300缩略图
- 元数据提取：获取新图片的宽度、高度、大小
- 数据库更新：更新文件路径、缩略图路径、尺寸信息
- 旧文件清理：删除原始文件和缩略图
- 后台AI分析：异步启动AI标签分析（不阻塞响应）

5.2.7 AI图片分析接口

接口URL	POST /api/images/:id/analyze
返回值类型	{ "message": "AI 标签生成成功", "tags": ["风景", "湖泊", "日落", "自然风光"] }
接口简介	调用通义千问视觉大模型分析图片内容，自动生成3-5个中文分类标签。

示意图：



接口调用核心实现 `aiService.js`:

```
// backend/services/aiService.js
const OpenAI = require('openai');
const fs = require('fs');

// 虽然用的是 OpenAI 的库，但 baseURL 指向阿里云服务器
const openai = new OpenAI({
  apiKey: process.env.DASHSCOPE_API_KEY,
  baseURL: "https://dashscope.aliyuncs.com/compatible-mode/v1",
});

const analyzeImage = async (imagePath) => {
  try {
    // 1. 读取本地图片转为 Base64 字符串供 AI 识别
```

```

const imageBuffer = fs.readFileSync(imagePath);
const base64Image = imageBuffer.toString('base64');

// 2. 发送请求给通义千问视觉大模型
const response = await openai.chat.completions.create({
  model: "qwen-vl-plus", // 这里根据百炼控制台的模型名填写
  messages: [
    {
      role: "user",
      content: [
        { type: "text", text: "给这张图3-5个中文分类标签，用逗号隔开，不要有废话。" },
        {
          type: "image_url",
          image_url: { url: `data:image/jpeg;base64,${base64Image}` },
        },
      ],
    },
  ],
});

// 3. 返回解析后的标签数组
const content = response.choices[0].message.content;
return content.split(/[, , ]/).map(tag => tag.trim());
} catch (error) {
  console.error("AI 分析具体报错:", error.message);
  return [];
}
};

module.exports = { analyzeImage };

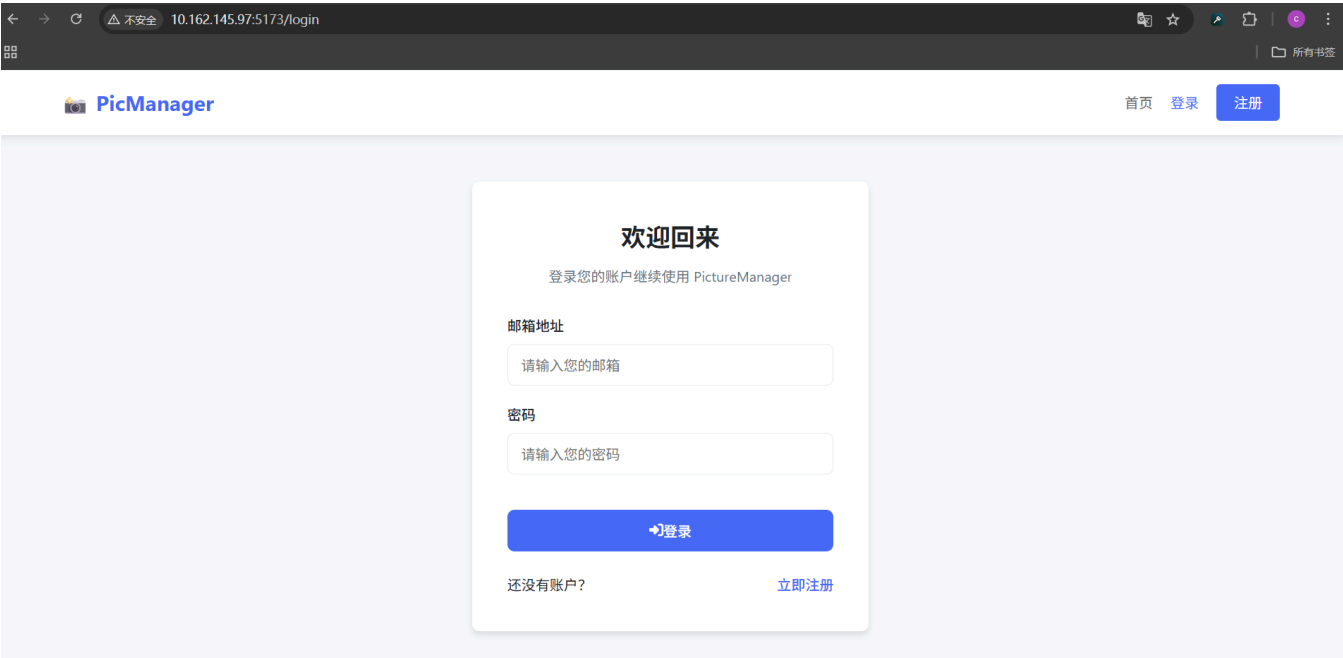
```

6.系统界面成品展示

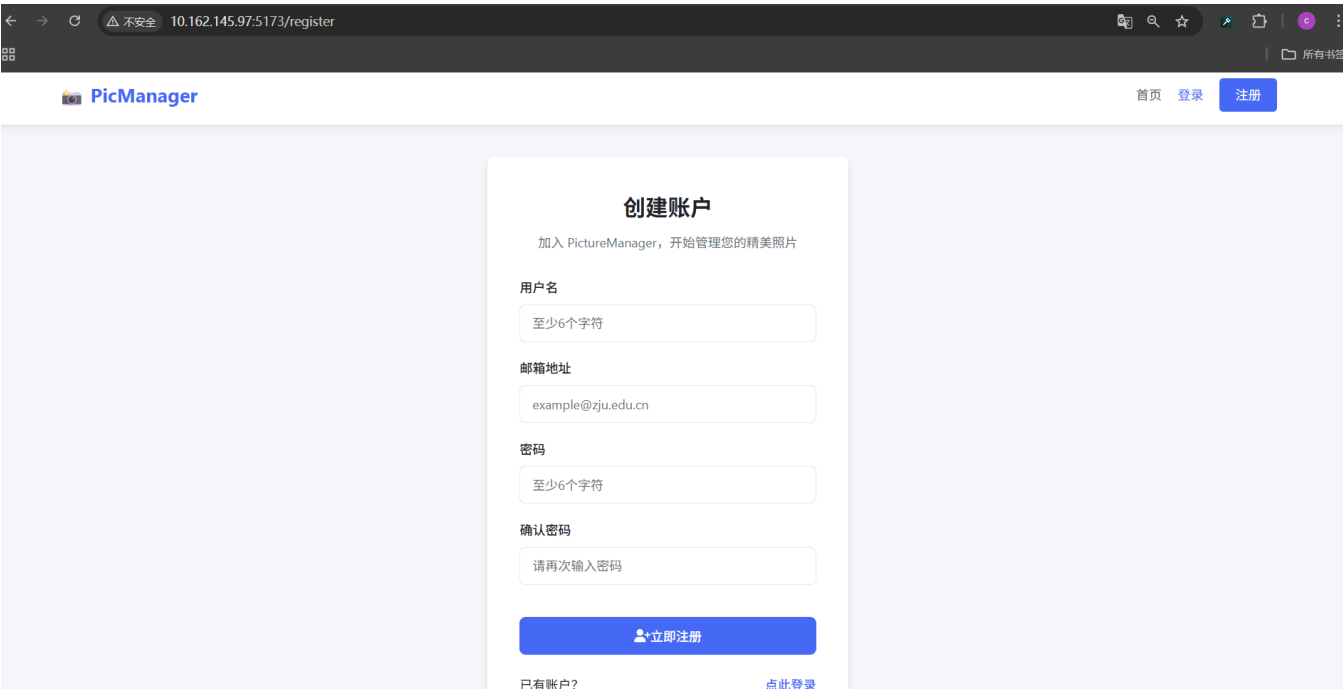
以下前端界面为最终版展示，详细见“测试报告”和“展示视频”

此处主要呈现界面，测试逻辑见“测试报告”。

6.1 登录界面



6.2 注册界面

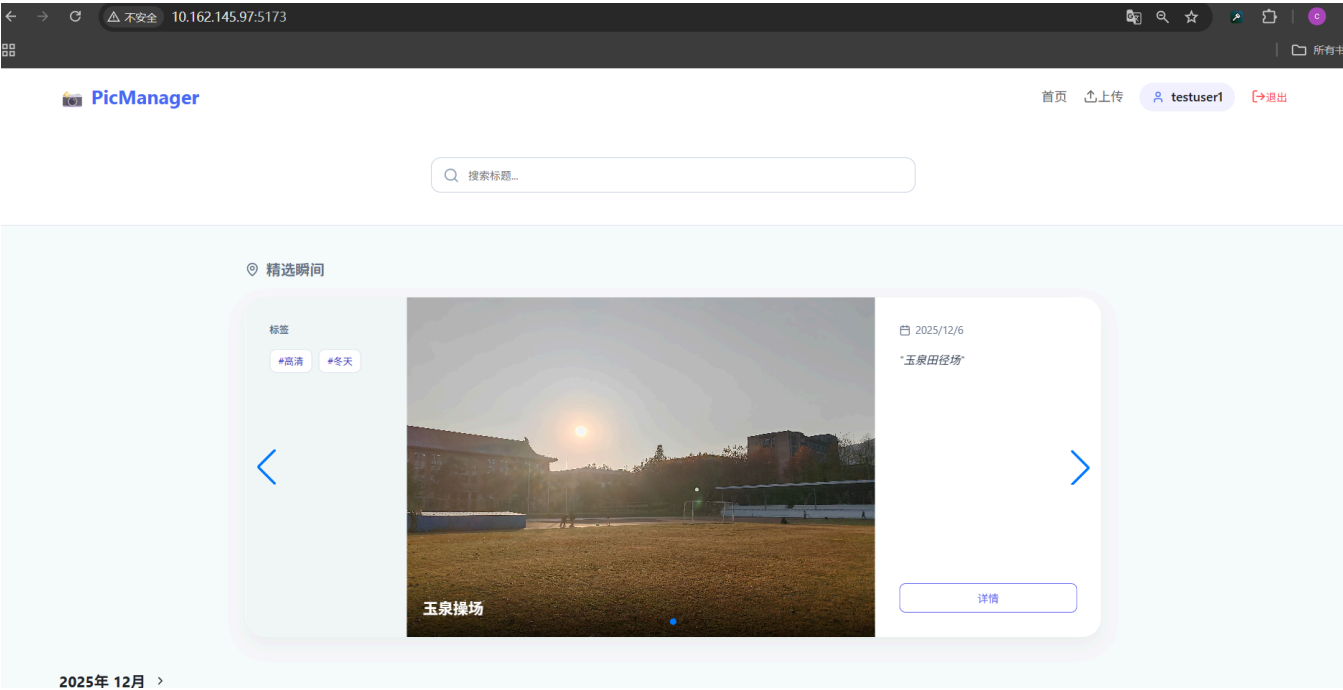


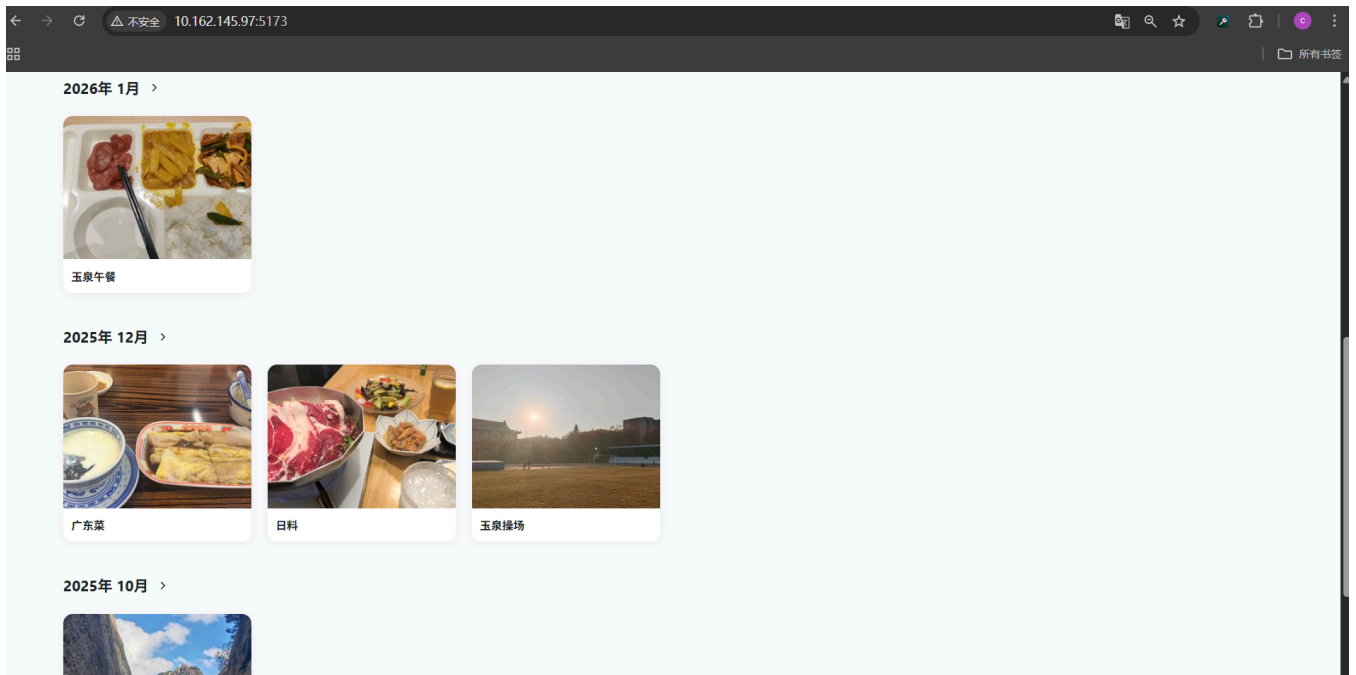
6.3 图片上传界面



6.4 首页和检索界面

首页页面较大，上半部分轮播图，下半部分缩略图分类排列。

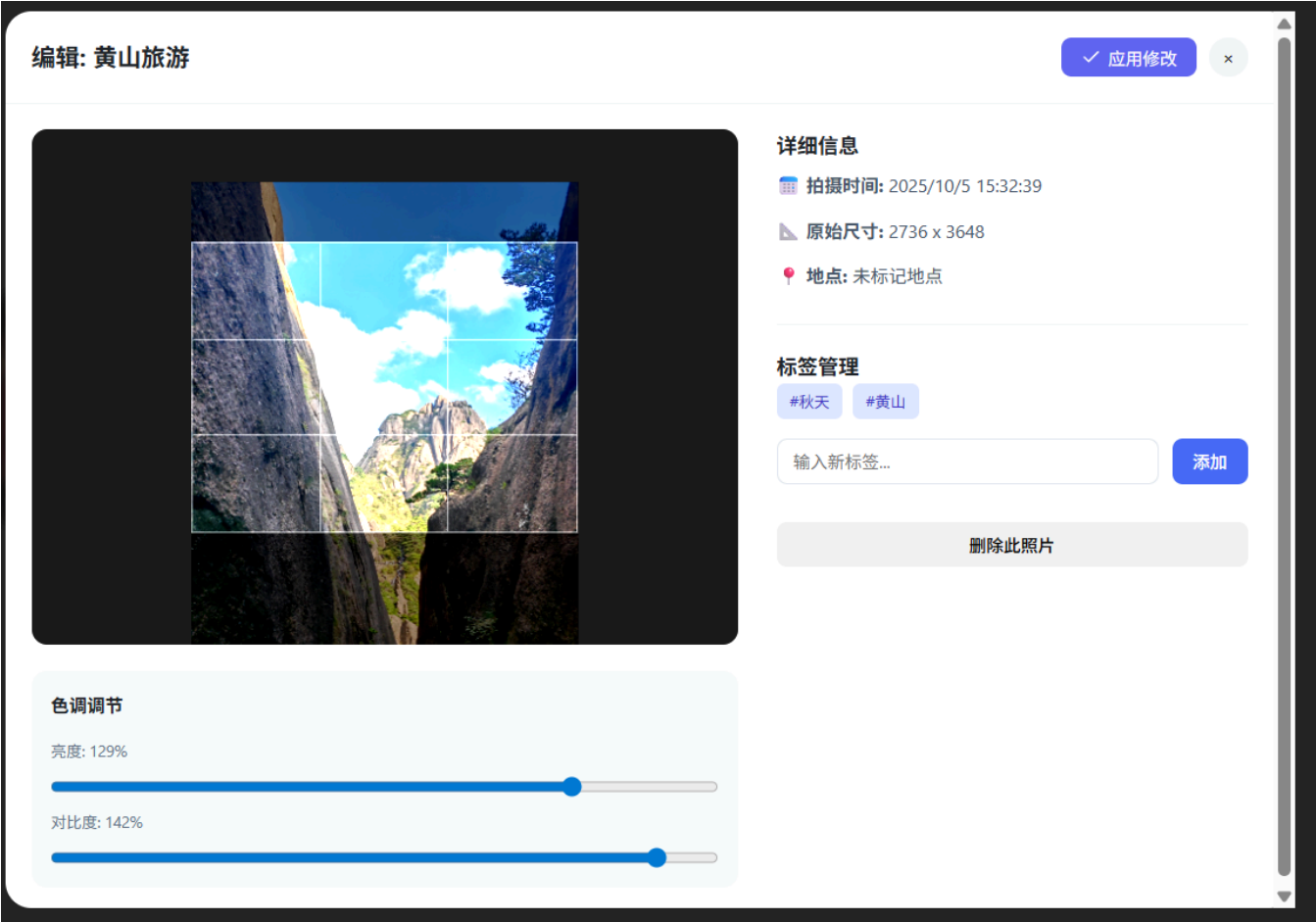




6.5 图片详情界面



6.6 图片编辑界面



7.总结与体会

项目实践过程记录见附录“实验过程记录”与git log信息。

Commits on Jan 3, 2026		
AI大模型标签, 完善前后端api设置	chrisyskg committed 5 hours ago	2324a38
Commits on Jan 2, 2026		
前后端图片编辑功能实现	chrisyskg committed yesterday	1a2c4f1
Commits on Dec 29, 2025		
首页优化界面, 轮播	chrisyskg committed 5 days ago	aab2eca
图片详情显示和人工标签添加	chrisyskg committed 5 days ago	5d9495a
前端简单首页和用户状态显示	chrisyskg committed 5 days ago	eb5c57c
Commits on Dec 27, 2025		
前端文件上传实现	chrisyskg committed last week	c72b2d7
前端注册界面交互实现	chrisyskg committed last week	092f11b
前端框架搭建和Login实现, 已成功登录	chrisyskg committed last week	1b7a242
后端查询删除图片和添加标签	chrisyskg committed last week	6dcabba
图片上传接口 (还无法免费测试)	chrisyskg committed last week	173b91f
Commits on Dec 26, 2025		
实现用户登录功能并集成 JWT Token 发放	chrisyskg committed last week	ba4bda0
authController通过了Thunder Client测试	chrisyskg committed last week	d58f017
initial commit-database connection success	chrisyskg committed last week	c6df15f

总体来说，项目始终个人独立探索，比较全面的完成了老师要求的任务，过程中后端接口以及前端编写都遇到了不少困难，最后也是赶工完成了。先前确实没有接触过React+Node.js的技术架构，一个人从零开始一步步把网站后端、前端搭建起来还是很有成就感，学到了很多架构设计以及前后端交互原理，亲身实践下来也是加深了，对B/S架构的理解，是一次非常有价值的实践作业。